



Complete Testing Guide

Step-by-step instructions with explanations for every test. UI tests + real AI agent tests, fully repeatable.

20 Tests End-to-end coverage	Why It Matters Explained for each test	Pass / Fail Exact indicators
--	--	--

LEGEND

- | | |
|------------------|----------------|
| Pass indicator | Fail indicator |
| Note | Reset / re-run |
| Why we test this | Navigate to |

Kynara UI Tests (Browser Only)

These tests require only a web browser — no coding or CLI. Run them in order; each builds on the previous.

Test 1

Register & Sign In

WHY This is the foundation of everything. Kynara enforces policies per organization, so before any agent can be registered or any policy created, we need a valid org identity. This test also verifies the password policy (minimum 12 characters) is enforced.

► kynaraai.com › Sign up

1. Open <https://kynaraai.com> in your browser and click the “Start free” button.
2. On the registration page, fill in all four fields:

Email *	your@email.com — use a real address you can access
Password *	Must be at least 12 characters. Example: <code>MyKynara2026!</code>
Display name *	Your full name
Organization name *	The name of your org, e.g. “Acme AI Labs”

3. Click “Create account”. Wait 2–3 seconds for the redirect.
4. Verify you land on the Dashboard page. Look for the org name in the switcher, the “Welcome to Kynara” checklist, and the 14-day trial badge.

✓ You see the Dashboard. Org name matches what you entered. Trial badge visible.

✗ If you stay on the registration page — check that the password is 12+ chars and the email is valid.

□ Password shorter than 12 characters shows a red validation error inline. This is intentional — we hardened the minimum in a recent security update.

🔄 **RESET** To re-run: Settings › Danger Zone › Delete organization. Or sign up with a new email.

WHY AI agents need a registered identity before Kynara can make decisions about them. The Agent ID is what your SDK passes as `subject_id`. Without this, the decision engine has no subject to evaluate and will default to deny. This test verifies identity creation and supervision mode assignment.

► Agents › New agent

1. Click “Agents” in the left sidebar. The Agents list page opens (empty for a new org).
2. Click the “+ New agent” button in the top-right corner.
3. Fill in the agent creation form:

Display name *	“CRM Test Agent” — a readable label for this agent
Description	“Testing agent for the Kynara testing guide” (optional)
Supervision mode *	Select “human_supervised” — enables the approval workflow
Daily action budget *	100 — limits how many actions this agent can take per day

4. Click “Create agent”. You are redirected to the agent detail page.
5. On the detail page, locate the **Agent ID** (a UUID like `550e8400-e29b-...`). Copy it — you will need it for every test that involves decisions.
6. Click the “Access Summary” tab. Verify it shows “No scopes granted” — this is correct because no roles have been assigned yet. **Test 3 fixes this.**

✓ Agent appears in the list. Detail page shows the Agent ID, supervision badge says “Human Supervised”, Access Summary is empty.

✗ “No agents found” — you may have navigated to the wrong org. Check the org switcher.

□ The daily action budget is a safety cap. When the agent hits 100 decisions in a day, subsequent calls return deny until midnight UTC resets the counter.

🔄 **RESET** From the agent detail page, click ... > Delete agent. Then re-run from step 1.

WHY Kynara evaluates every request through **two gates, in order**. First the **RBAC gate**: the agent must hold a role that *grants* a scope matching the requested action. **Only then** are policies (allow / deny / require_approval) evaluated. An allow policy **cannot** grant a scope on its own — a role grants the scope, and the policy decides *when* it applies. In Test 2 the agent's Access Summary showed “No scopes granted”, so right now every decision for it returns deny with the reason `no role grants scope 'crm:contacts.read'` — before any policy is even considered. This test creates a role granting `crm:contacts.read`, assigns it to the agent, and verifies the scope appears in the Access Summary. **Skip this and the Simulator (Test 6) and every SDK test (17–20) will fail at the RBAC gate.**

► Roles › New Role • then Agents › [your agent] › Roles › Add role

Part A — Create the role

- Click “Roles” in the left sidebar. Click “+ New Role”.
- Fill in the role form:

Display name *	“CRM Reader” — the Slug auto-fills to <code>crm-reader</code>
Description	“Read-only access to CRM contacts” (optional)
Scopes *	Add <code>crm:contacts.read</code> . Either search the scope catalog and click it, or type it into the “custom scope” box and click Add . A chip <code>crm:contacts.read</code> should appear in the selected list.

- Click “Save”. The role appears in the Roles list showing slug `crm-reader` and 1 scope.

Scope `crm:contacts.read` . `crm:*` , `crm:contacts.*` , * all `crm:contacts.read` .
 notation Wildcards or satisfy
 uses a are
 colon allowed
 after and
 the tool match via
 domain: the
 engine:

Part B — Assign the role to your agent

- Click “Agents” > open your “CRM Test Agent”. Scroll to the **Roles** section (“users assigned to operate this agent with a specific role”).
- Click “+ Add role”. In the dialog:

User *	Select your own user — the agent acts on behalf of this user
Role *	Select “CRM Reader”
Expires	Leave blank (Never) for testing

- Click “Assign”. The assignment row appears: your user, role “CRM Reader”, Expires “Never”.
- Click the “Access Summary” tab. It should now list `crm:contacts.read` under granted scopes — replacing the “No scopes granted” you saw in Test 2.

✓ Role “CRM Reader” `crm:contacts.read` . The agent's Access Summary now shows `crm:contacts.read` .

✗ Access Summary still shows “No scopes granted” — the assignment didn't save, or the role has no scopes. Re-open the role and confirm the scope chip is present, then re-add the assignment.

☐ **#1 cause of** the *literally* . A role `crm.contacts.read` (all **not** satisfy `crm:contacts.read` (colon). **Action**
an engine granting dots) an
unexpected compares will an
deny: scope of
strings

Keep
the
role
scope,
the
policy

 **RESET** Agents > your agent > Roles > click remove on the assignment. To delete the role: Roles > ... > Delete (non-system roles only).

WHY Policies are the rules Kynara evaluates to produce a decision once the RBAC gate (Test 3) has passed. Without policies, every decision returns “deny” by default (fail-closed). We create three policies to exercise all three outcomes: allow, deny, and require_approval. We also verify priority ordering — lower priority numbers are evaluated first, and the first matching policy wins.

► Policies › New policy

Policy A — the ALLOW case (business hours)

1. Click “Policies” in the sidebar. Click “+ New policy”.
2. Fill in Policy A:

Display name *	“Allow CRM reads — business hours”
Priority *	100 (evaluated second, after the deny rule)
Effect *	allow
Actions (Scopes) *	<code>crm:contacts.read</code>
Resource types *	<code>crm.contact</code>
Condition (JSON) *	<code>{"op": "time_between", "args": ["ctx.context.time", "09:00", "18:00"]}</code>

3. Toggle “Enabled” ON. Click “Save policy”.

Policy B — the REQUIRE_APPROVAL case (off-hours)

1. Click “+ New policy” again. Fill in Policy B:

Display name *	“Require approval — CRM reads off-hours”
Priority *	200 (evaluated third, after the geo-deny)
Effect *	require_approval
Actions *	<code>crm:contacts.read</code>
Resource types *	<code>crm.contact</code>
Condition *	<code>{"op": "not", "args": [{"op": "time_between", "args": ["ctx.context.time", "09:00", "18:00"]}]}</code>

2. Toggle Enabled ON. Click Save.

Policy C — the DENY case (geo-block)

1. Click “+ New policy” again. Fill in Policy C:

Display name *	“Deny — blocked countries”
Priority *	50 (evaluated FIRST — lower number = higher priority)
Effect *	deny
Actions *	(leave empty — applies to ALL actions)
Resource types *	(leave empty)
Condition *	<code>{"op": "in", "args": ["ctx.context.ip_country", ["CN", "KP", "RU"]]}</code>

2. Toggle Enabled ON. Click Save.

3. Return to the Policies list. Verify all three appear, sorted by priority: 50 → 100 → 200.

✓ Three policies at priorities 50, 100, 200. Each shows the correct effect badge (deny / allow / require_approval).

✗ Condition shows a red validation error — check JSON syntax. Curly braces and quotes must be correct.

□ Priority ordering matters critically: Policy C (deny, priority 50) is evaluated BEFORE Policy A (allow, priority 100). A request from CN hits Policy C and is denied even during business hours — the geo-block cannot be bypassed by time.

🔄 **RESET** Delete all three policies from the Policies list. Re-run from step 1.

Test 5

Create an API Key

WHY The decisions endpoint uses API keys, not user JWTs. This separation is intentional: agents (automated systems) authenticate with scoped API keys, while humans authenticate with JWTs. The scope `decisions.check` limits this key to only calling the decision endpoint — it cannot create policies, access billing, or read audit logs.

► Settings › API Keys › New API Key

1. Click “Settings” in the sidebar. Click the “API Keys” tab.
2. Click “+ New API Key”. Fill in:

Name *	“Test SDK Key” — a descriptive label
Scopes *	Select <code>decisions.check</code> from the dropdown list

3. Click “Create”. A modal appears showing the full API key.
4. **IMPORTANT:** Copy the secret immediately. It starts with `sk_live_` and is 72 characters long. This is the ONLY time it is shown in full.
5. Click “Done”. The key now appears in the list showing only the first 12 chars.
6. To verify: click the key row and check that `decisions.check` appears in the Scopes column.

✓ Key appears in list with prefix `sk_live_`. Scope shows `decisions.check`. The full secret was visible in the creation modal.

✗ “Invalid scopes” error — make sure you selected `decisions.check` from the dropdown, not typed it manually.

□ If you lose the secret without copying it, click Revoke and create a new key. There is no way to retrieve a key secret after the modal is closed.

🔄 **RESET** Click Revoke next to the key. Create a new one for the next run.

WHY Now that the agent holds the `crm:contacts.read` scope (Test 3) and the policies exist (Test 4), we use the built-in Simulator to verify they produce the correct outcomes. This is the safest way to test policy logic — no actual agent runs, no quota consumed. It also teaches how context fields (time, ip_country) affect decisions.

► Policies › [open any policy] › Simulator panel

Test 6a — the ALLOW case

- Click “Policies” › “Allow CRM reads — business hours” (Policy A).
- Scroll past the condition editor to the “Simulator” panel. Fill in:

Subject type *	agent
Subject ID *	Paste your Agent ID from Test 2
Action *	<code>crm:contacts.read</code>
Resource type *	<code>crm.contact</code>
Resource ID *	<code>contact_001</code>
Context (JSON) *	<code>{"time": "10:30", "ip_country": "US"}</code>

- Click “Simulate”. Read the result panel.

✓ Effect: **allow** . Reason: “Matched policy: Allow CRM reads — business hours”. The trace shows Policy C (deny) was checked first but did not match (US is not blocked), then Policy A matched.

✗ Effect: deny, reason `crm:contacts.read` “no role grants scope” — the agent is missing its role assignment. Go back and complete **Test 3** (check the Access Summary shows the scope).

Test 6b — the REQUIRE_APPROVAL case

- Change Context to `{"time": "22:30", "ip_country": "US"}` and click Simulate.

✓ Effect: **require_approval** . Matched Policy B (off-hours). An `approval_id` is created if this were a real decision.

Test 6c — the DENY case (geo-block)

- Change Context to `{"time": "10:30", "ip_country": "CN"}` and click Simulate.

✓ Effect: **deny** . Matched Policy C — blocked countries. The trace shows it stopped at priority 50 without evaluating 100 or 200.

Test 6d — the DEFAULT DENY case (no policy match)

- Change Action to `payments.refund.issue` (no policy covers this) and click Simulate.

✓ Effect: **deny** . Reason: “No policy matched — default deny”. This proves Kynara is fail-closed.

□ The Simulator does NOT create audit log entries or consume your monthly decision quota. Use it freely for policy testing before deploying changes.

🔄 **RESET** Simulator state is not persisted. Re-run any combination at any time.

WHY When a policy returns `require_approval`, the agent must pause and wait for a human to decide. This is Kynara's core human-in-the-loop feature. We test the full lifecycle: approval appears in the queue, reviewer inspects details, approves or rejects, status updates, and the audit trail is written.

► Approvals

1. First, create a pending approval: run Test 6b (the off-hours Simulator call). The Simulator creates a real pending approval when the effect is `require_approval`.
2. Click "Approvals" in the sidebar. A badge count shows pending items.
3. Locate the pending approval. Verify: Agent = "CRM Test Agent", Action = `crm:contacts.read`, Status = PENDING (amber), Expires ≈ 24 h from now.
4. Click the row to open the detail view. Review the risk score, the context (`time: 22:30`, `ip_country: US`), and the matched policy ("Require approval — CRM reads off-hours").
5. In the "Review note" field, type: "Approved — off-hours access verified for testing".
6. Click the green "Approve" button. Verify the badge changes to "APPROVED" (green).
7. Check the audit trail: Audit log > find the `approval.approved` event. It should show your user ID as reviewer.
8. Now test rejection: run Test 6b again to create a second pending approval.
9. Open it, type a rejection note "Rejected — test run", and click the red "Reject" button. Verify status changes to "REJECTED".

✓ Two resolved approvals — one `approval.approved` and `approval.rejected` with your name and notes. approved, one rejected. Audit log shows

✗ "No approvals found" — make sure you ran Test 6b which uses time 22:30 (off-hours). The 10:30 context produces `allow`, not `require_approval`.

□ Approvals expire after 24 hours if not acted upon. Expired approvals cannot be approved retroactively.

🔄 **RESET** Approvals are immutable once resolved. Re-run Test 6b to create fresh pending approvals.

WHY The audit log is Kynara's compliance backbone. Every decision, approval, login, and admin action is appended as a hash-chained entry — like a blockchain. If anyone modifies or deletes a past record (even a DB admin), the chain breaks and is detectable. This test verifies the chain is intact and that your earlier tests appear correctly.

► **Audit log**

1. Click “Audit log” in the sidebar. Verify recent events appear: `login.success` (Test 1), `agent.created` (Test 2), `agent.assigned` (Test 3), `decision.recorded` (Test 6), `approval.approved` and `approval.rejected` (Test 7).
2. Click any entry. Verify the detail panel shows: Event type, Actor, Outcome, Payload (JSON context), Hash (SHA-256), and Prev hash (the chain link).
3. Click “Verify Chain” (top-right). Wait 2–3 seconds and read the result.
4. Test filtering: use the “Event type” dropdown to select `decision.recorded`. Then try the “Outcome” filter with “deny”.
5. Test CSV export: click “Export CSV” and verify the file contains the same events shown on screen.

✓ “Chain intact — N events verified, no tampering detected.” N matches your total event count.

✗ “Chain broken at event X” — a record was modified after being written. In a fresh test environment, contact support immediately.

□ The audit log is append-only. You cannot delete individual entries — this provides the tamper-evidence guarantee.

🔄 **RESET** Audit log is immutable. Each run adds new entries. No cleanup needed.

WHY Sometimes an agent needs temporary elevated access for an emergency — without permanently modifying any policy or role. JIT grants widen the agent's effective scope for a set duration, then auto-expire. Every grant requires a justification and optionally a ticket link, creating a clear audit trail.

► **Agents** › **[your agent]** › **JIT Grants** tab

1. Click “Agents” › your “CRM Test Agent” › the “JIT Grants” tab.
2. Click “+ New Grant”. Fill in:

Scope *	<code>payments:refund.issue</code> — a scope your agent does NOT currently have
Duration (minutes) *	5 — short for testing; production uses 60–120
Justification *	“Emergency test: verifying JIT grant workflow”
Ticket URL	https://jira.example.com/TEST-001 (optional)

3. Click “Create Grant”. The grant appears with an expiry countdown.
4. Click the “Access Summary” tab. Verify `payments:refund.issue` now shows “(via JIT grant)” with an “Expires in ~5 min” badge.
5. To test early revocation: JIT Grants tab › “Revoke”. Verify the scope immediately disappears from Access Summary.
6. Check the Audit log for `jit_grant.created` and `jit_grant.revoked` entries.

✓ Grant appears with correct scope and expiry. Access Summary shows the scope. After revoke/expiry, the scope is gone. Two audit entries appear.

✗ “Scope not found” in Access Summary — ensure the grant is still active (not expired) and refresh the page.

□ JIT grants override policy evaluation for their duration. An agent with a JIT grant for `payments:refund.issue` can call that action even if no allow policy exists for it.

🔄 **RESET** Click Revoke if the grant is still active. Grants expire automatically. No cleanup needed.

WHY Policy changes are risky. A policy that looks correct in the Simulator might have unintended consequences at scale. Policy Replay simulates a proposed change against real historical decisions — showing exactly how many would flip outcomes — before a single live agent is affected.

► Policies › [open Policy A] › Replay tab

1. Click “Policies” › “Allow CRM reads — business hours” (Policy A).
2. Scroll to the “Replay against history” section.
3. Without saving, tighten the window in the editor: change 09:00 to 10:00 and 18:00 to 17:00. **Do NOT click Save** — you are just testing impact.
4. Set the Lookback window to “7 days”. Click “Run Replay”. Wait 2–5 seconds.
5. Read the results: Total events evaluated, Flipped count, allow → require_approval, allow → deny.
6. Click any affected decision ID to inspect the full context of a decision that would flip.
7. Click “Cancel” or navigate away WITHOUT saving — the policy condition is unchanged.

✓ Results show `total_events` ≥ the number of decisions made in Tests 6–7. Flip counts reflect decisions made with times in the 09:00–10:00 or 17:00–18:00 windows.

□ If `total_events` is 0, not enough history exists yet. Run more Simulator calls in Test 6 first, then re-run the replay.

↺ **RESET** Replay is read-only — no state changes. Re-run any time.

WHY Webhooks let your systems react to Kynara events in real time — posting approvals to Slack, firing PagerDuty alerts, feeding your SIEM. This test verifies deliveries are HMAC-signed (so your endpoint can verify they came from Kynara) and that the payload contains the expected fields. We use `webhook.site` as a free, no-setup receiver.

► **Settings > Webhooks > + New Webhook**

1. Open a new tab at **`https://webhook.site`** and copy “Your unique URL”.
2. In Kynara: Settings > Webhooks > “+ New Webhook”. Fill in:

URL *	Paste your <code>webhook.site</code> URL
Events *	Select <code>decision.allowed</code> , <code>decision.denied</code> , <code>decision.approval_requested</code> , <code>agent.killed</code>

3. Click “Save”. A “secret” is shown once — copy it (format `whsec_...`).
4. Click “Send Test Event”. Switch to `webhook.site` — a delivery appears within 2 seconds.
5. Inspect the delivery. Verify the `X-Kynara-Signature` header starts with `sha256=v1,`, Content-Type is `application/json`, and the body has `event_type`, `payload`, `event_id`.
6. Run Test 6a (allow). Check `webhook.site` — a `decision.allowed` delivery arrives. Then run Test 6c (deny from CN) — a `decision.denied` delivery arrives.

✓ Deliveries appear within 2 seconds. Signature `sha256=v1,`. Each decision type from Test 6 creates a corresponding delivery.

✗ No delivery after 10 seconds — the `webhook.site` URL may have expired. Get a new one and update the endpoint URL.

□ The signature format is `sha256=v1,` — note the “v1,” prefix. This `sha256=` format. Verify your signature-check code uses the new format.

🔄 **RESET** Delete the webhook endpoint from Settings > Webhooks. Get a fresh `webhook.site` URL for the next run.

WHY When an agent triggers a `require_approval` decision, the team shouldn't have to open Kynara to review it. Slack integration sends an interactive message to the reviewer's channel. Clicking Approve or Reject in Slack updates Kynara instantly.

Part A — one-time Slack app setup

1. Go to <https://api.slack.com/apps> and sign in.
2. Click "Create New App" > "From scratch". Name it "Kynara Approvals". Select your workspace.
3. Under Features > "OAuth & Permissions" > Bot Token Scopes, add `chat:write` and `chat:write.public`.
4. Click "Install to Workspace". Authorize. Copy the Bot User OAuth Token (`xoxb- ...`).
5. Basic Information > App Credentials > copy the "Signing Secret".
6. Interactivity & Shortcuts > toggle Interactivity ON. Set Request URL to <https://kynaraai.com/api/v1/integrations/slack/callback>.
7. In Slack, right-click the target channel > View channel details > copy the Channel ID (starts with `C`).

Part B — configure in Kynara

1. Click "Integrations" in the Kynara sidebar.
2. In the Slack card, fill in the Bot Token, Signing Secret, and Channel ID. Toggle Enabled ON. Click "Save integration".
3. Click "Send test notification". Check your Slack channel.

✓ A Block Kit message appears in Slack with: Approval Required, agent name, action, policy matched, expiry, and two buttons — Approve and Reject.

Part C — full approval-from-Slack test

1. Run Test 6b (off-hours Simulator call). A real Slack notification fires within 1–2 seconds.
2. In Slack, click "Approve". In the confirmation dialog, click "Yes, approve".
3. The Slack message updates to "Approved by @yourname".
4. In Kynara > Approvals, verify the status is "approved" and the reviewer shows your Slack username.

✓ Slack message appears. Clicking Approve `approval.approved` with `slack_integration` .
updates Kynara status. Audit log shows source

❑ The Interactivity Request URL must be publicly accessible. If testing locally, use ngrok to expose your local server.

🔄 **RESET** Slack configuration persists per org. No reset needed between runs.

WHY For teams using Microsoft 365 / Teams instead of Slack, Kynara posts Adaptive Cards to Teams channels when approvals are needed. The card includes deep-link buttons to the Kynara approval page so reviewers can resolve in one click.

Part A — one-time Teams webhook setup

1. In Teams, navigate to the target channel. Click the “...” next to the channel name.
2. Click “Connectors”. Search “Incoming Webhook” > Configure.
3. Name it “Kynara Approvals” (optionally upload a logo). Click “Create”.
4. Copy the generated webhook URL (starts with `https://...webhook.office.com/...`).

Part B — configure in Kynara and test

1. In Kynara > Integrations, find the Teams card. Paste the webhook URL. Toggle Enabled ON. Save.
2. Click “Send test notification”. Switch to Teams.
3. Run Test 6b to trigger a real approval notification.
4. In Teams, click “Approve in Kynara” — this opens your browser to the approval page. Click Approve, then return to Teams.

✓ An Adaptive Card appears with an Approval Required header, a fact table (Agent, Action, Policy, Expires), and three buttons: Approve in Kynara, Reject in Kynara, View details.

❑ Teams Incoming Webhooks do not support interactive buttons natively — they use `Action.OpenUrl` which opens the browser. For fully inline approve/reject, you would need Power Automate (advanced).

🔄 **RESET** Teams config persists per org. No reset needed.

WHY Some Kynara events need to wake someone up at 3am — agent killed, audit chain broken, or anomalous behavior. PagerDuty integration pages on-call engineers immediately. This test verifies the kill switch fires a PagerDuty incident correctly.

Part A — get a PagerDuty routing key

1. Log in to PagerDuty at <https://app.pagerduty.com>.
2. Services > select or create a service > Integrations tab > “+ Add an integration”.
3. Select “Events API v2” > Add. Copy the 32-hex-character integration (routing) key.

Part B — configure and test

1. In Kynara > Integrations, find the PagerDuty card. Paste the routing key. Toggle Enabled ON. Save.
2. To trigger an alert: Agents > your “CRM Test Agent” > click the red “Kill switch” button.
3. In the confirmation dialog click “Confirm kill”.
4. Switch to PagerDuty. Within 5–10 seconds a new incident appears: title “Kynara: Kill switch activated on CRM Test Agent”, severity critical, with Agent ID, org ID, and a link to the audit log.
5. In Kynara > Agents, the agent shows as disabled (grey “Killed” badge). Re-enable it via “Enable agent”.

✓ PagerDuty incident fires within 10 seconds. Agent shows as killed in Kynara. Audit log shows `agent.killed` .

❑ The kill switch cannot be undone via PagerDuty — you must re-enable in Kynara. The audit trail records both the kill and the re-enable.

🔄 **RESET** Re-enable the agent. Resolve or acknowledge the PagerDuty incident. Config persists.

WHY After running the approval workflow in Tests 6–7, Kynara computes analytics over the approval history. This test verifies those metrics and helps you understand review patterns — which agents generate the most approvals, how quickly reviewers respond, and the approval/rejection rate.

► **Approval Analytics (sidebar, under Approvals)**

1. Click “Approval Analytics” in the sidebar (under the “Approvals” section / Governance group).
2. Set the time window to “Last 30 days”.
3. Verify the KPI cards reflect Tests 6–7:

Total	≥ 2 (the two approvals from Test 7)
Approved	1 (the one you approved in Test 7)
Rejected	1 (the one you rejected in Test 7)
Approval rate %	50% (1 of 2 resolved)
Avg. resolution time	minutes it took you to approve/reject

4. Scroll to “Top agents by approvals” — your “CRM Test Agent” appears.
5. Check “Top actions” — `crm:contacts.read` should be the top entry. The daily volume chart shows a bar for today.
6. Switch the window to “Last 7 days” — counts stay the same (all tests run within the week).

✓ Total ≥ 2. Approval rate 50%. CRM Test Agent appears in top agents. Daily chart shows a bar for today.

❑ If Avg. resolution time is null, no approvals were resolved yet. Complete Test 7 first, then return.

🔄 **RESET** Analytics are computed live. No reset needed.

WHY Guardrails are your last line of defense against runaway agents. If an agent sends too many high-severity events in a short window (which can indicate prompt injection or a compromised agent), the guardrail automatically revokes access without waiting for a human. This test verifies threshold detection and auto-revocation.

► [Guardrails](#) > [+ New Integration](#) > [Rules](#) > [+ New Rule](#)

1. Click “Guardrails” > “+ New Integration”. Name it “Test Guardrail” > Create. Copy the generated Webhook URL.
2. Click the “Rules” tab > “+ New Rule”. Fill in:

Threshold *	3 events — trigger after 3 (low, for testing)
Time window *	5 minutes
Severity filter *	high
Action *	revoke — immediately disable the agent

3. Click “Save Rule”.
4. Post test events to breach the threshold: use API Explorer (POST /guardrails/events) or send 3 POSTs to the guardrail webhook URL with body


```
{"agent_id": "YOUR_AGENT_ID", "event_type": "tool_call_blocked", "severity": "high"}.
```
5. After 3 events: Agents > your agent > refresh. Verify it is disabled (grey “Killed” badge).
6. Check the Audit log for `agent.killed` with reason “guardrail: threshold exceeded”. Then re-enable the agent.

✓ Agent shows as killed after 3 high-severity events. Audit log shows the guardrail trigger. Agent re-enables successfully.

□ The 3-event threshold is only for testing. Production guardrails typically use 10–50 events over 5–15 minutes to avoid false positives.

🔄 **RESET** Re-enable the agent. Delete the Test Guardrail from the Guardrails page.

PART 2

Real AI Agent Tests (Google Colab — Free)

Connect a real Python agent to Kynara via the SDK in Google Colab. No local install. Complete Tests 1–5 first to have your credentials ready.

What you need

Where to get it

Google account	Free — for Google Colab access
Kynara base URL	https://kynaraai.com
API key (sk_live_...)	Created in Test 5
Agent ID (UUID)	Copied in Test 2
Hugging Face token (Test 20)	Free at huggingface.co > Settings > Access Tokens

One-time Colab setup (run once, then skip for subsequent tests)

1. Open <https://colab.research.google.com>. Click "+ New notebook".
2. In the first cell, paste and run:

```
!pip install kynara-sdk -q
```

3. In the next cell, set credentials (replace the placeholders):

```
import os
os.environ["KYNARA_BASE_URL"] = "https://kynaraai.com"
os.environ["KYNARA_API_KEY"] = "sk_live_YOUR_KEY_FROM_TEST_5"
os.environ["KYNARA_AGENT_ID"] = "YOUR_AGENT_UUID_FROM_TEST_2"
```

4. Run both cells. No errors means setup succeeded.

WHY The most fundamental integration test. It verifies the Kynara SDK communicates with the decision engine, that your API key and agent ID are valid, that the agent's role grants `crm:contacts.read` (Test 3), and that all three outcomes work end-to-end from code.

Platform: Google Colab (free) — continue from the one-time setup above.

1. In a new Colab cell, paste this block and run it:

```
from kynara_sdk import Kynara
kynara = Kynara.from_env()
print(f"Connected. Agent ID: {kynara.agent_id[:8]}...")

# --- allow: business hours, allowed country ---
d1 = kynara.check(
    subject=("agent", kynara.agent_id),
    action="crm:contacts.read",
    resource={"type": "crm.contact", "id": "c_001", "attrs": {}},
    context={"time": "10:30", "ip_country": "US"},
)
assert d1.effect == "allow", f"Expected allow, got {d1.effect}"
print(f"[ALLOW] effect={d1.effect} policy={d1.matched_policy_id[:8]}...")

# --- require_approval: off-hours ---
d2 = kynara.check(
    subject=("agent", kynara.agent_id),
    action="crm:contacts.read",
    resource={"type": "crm.contact", "id": "c_001", "attrs": {}},
    context={"time": "22:30", "ip_country": "US"},
)
assert d2.effect == "require_approval", f"Expected require_approval, got {d2.effect}"
print(f"[APPROVAL] effect={d2.effect} approval_id={d2.approval_id[:8]}...")

# --- deny: blocked country ---
d3 = kynara.check(
    subject=("agent", kynara.agent_id),
    action="crm:contacts.read",
    resource={"type": "crm.contact", "id": "c_001", "attrs": {}},
    context={"time": "10:30", "ip_country": "CN"},
)
assert d3.effect == "deny", f"Expected deny, got {d3.effect}"
print(f"[DENY] effect={d3.effect} reason={d3.reason}")
print("All 3 decision outcomes verified!")
```

2. In Kynara > Audit log, verify 3 new `decision.recorded` events appear.

✓ All three print statements run. No AssertionError. Audit log shows 3 new entries.

✗ AssertionError on d1 (got deny): either your policies from Test 4 are disabled, or the `crm:contacts.read` . agent lost its role assignment from Test 3 — check Access Summary shows

✗ 401 Unauthorized: API key is wrong. Check `KYNARA_API_KEY` matches your `sk_live_` key from Test 5.

✗ 404 Not Found: Agent ID is wrong. Re-copy it from Kynara > Agents > your agent detail page.

🔄 **RESET** Re-run the cell freely. Each run creates 3 new audit entries.

WHY In production, agents wrap tool functions with `@permission_required`. This test proves the critical guarantee: when Kynara denies a request, the tool function body NEVER executes. The side effect is prevented before it happens — fundamentally different from checking permissions after the fact.

Platform: Google Colab — continue from the previous test.

1. In a new Colab cell, paste and run:

```
from kynara_sdk import Kynara, permission_required, PermissionDenied
from kynara_sdk.context import set_context
kynara = Kynara.from_env()
execution_log = [] # tracks whether tool body ran

@permission_required(
    "crm:contacts.read",
    resource_arg="contact_id",
    resource_type="crm.contact",
)
def read_contact(contact_id: str) -> dict:
    # This line only runs if Kynara returns allow
    execution_log.append(f"EXECUTED for {contact_id}")
    return {"id": contact_id, "name": "Alice Smith"}

# Case 1: allow (business hours, US)
set_context({"time": "10:30", "ip_country": "US"})
result = read_contact("c_001")
print(f"Case 1 (allow): {result}")
print(f" Tool ran: {len(execution_log)} time(s)")

# Case 2: deny (blocked country)
set_context({"time": "10:30", "ip_country": "CN"})
try:
    read_contact("c_002")
except PermissionDenied as e:
    print(f"Case 2 (deny): BLOCKED -- {e.decision.reason}")
    print(f" Tool ran: {len(execution_log)} time(s) -- still just 1!")

assert len(execution_log) == 1, f"Tool ran {len(execution_log)} times, expected 1"
print("Tool body blocked on deny -- side effect prevented")
```

2. In Kynara > Audit log, verify 2 decision entries: one allow, one deny.

✓ Case 1 returns the `PermissionDenied` . `execution_log` has exactly 1 entry (only the allow contact. Case 2 raises

□ This proves the “no side effects on deny” guarantee. The `c_002` — the function returned immediately at the CRM was never queried for decorator boundary.

🔄 **RESET** Re-run freely. Each run adds 2 audit entries.

WHY When an agent hits `require_approval`, it must pause and surface an approval URL to a human — and only continue after the human decides. This test proves the full loop: agent code calls Kynara → receives `approval_id` → human reviews in the UI → agent verifies the outcome.

Platform: Google Colab + Kynara UI.

1. In a new Colab cell, paste and run:

```
from kynara_sdk import Kynara, ApprovalRequired
import requests, os
kynara = Kynara.from_env()
approval_id = None

print("Agent: requesting off-hours CRM access...")
try:
    kynara.enforce(
        subject="agent", kynara.agent_id,
        action="crm:contacts.read",
        resource={"type": "crm.contact", "id": "c_003", "attrs": {}},
        context={"time": "23:00", "ip_country": "US"}, # off-hours -> approval
    )
    print("Agent: access granted -- proceeding")
except ApprovalRequired as e:
    approval_id = e.decision.approval_id
    print("Agent: PAUSED -- waiting for human approval")
    print(f"  Approval ID: {approval_id}")
    print(f"  Review URL: https://kynaraai.com/app/approvals/{approval_id}")
    print(">> ACTION REQUIRED: open the URL and click Approve or Reject")
```

2. Copy the printed Review URL. Open it in your browser (logged into Kynara).
3. In the Approvals page, enter a note "Approved via human-in-the-loop test" and click Approve.
4. Return to Colab. In a new cell, paste and run:

```
headers = {"Authorization": f"Bearer {os.environ['KYNARA_API_KEY']}" }
r = requests.get(
    f"https://kynaraai.com/api/v1/approvals/{approval_id}",
    headers=headers,
)
status = r.json()["status"]
print(f"Approval status: {status}")
print(f"Reviewed at: {r.json().get('reviewed_at', 'null')}")
assert status == "approved", f"Expected approved, got {status}"
print("Human approved -- agent can now proceed")
```

✓ First cell prints `approval_id` and URL. After approving in Kynara, the second cell prints status: PAUSED with an approved and a timestamp. Assertion passes.

✗ status = pending: you have not approved it in the Kynara UI yet. Complete the approve step first.

🔄 **RESET** Resolved approvals are immutable. Re-run the first cell to create a new pending approval.

WHY Tests 17–19 used the SDK directly. This test wires Kynara into a real LLM-powered agent that autonomously decides which tools to call. The `KynaraCallbackHandler` intercepts every tool call before it runs. This proves Kynara sits **outside the LLM's trust boundary**: even if the LLM decides to call a restricted tool, Kynara blocks it — prompt injection cannot bypass the policy engine.

Platform: Google Colab + free Hugging Face account.

Step 1 — get a free Hugging Face token

1. Go to <https://huggingface.co> and create a free account / log in.
2. Profile > Settings > Access Tokens > “New token”. Name it “kynara-test”, select “Read”, click Generate.
3. Copy the token (starts with `hf_`).

Step 2 — install dependencies in Colab

```
!pip install kynara-sdk langchain langchain-community huggingface_hub -q
```

Step 3 — define Kynara-gated tools

```
import os
from langchain.tools import tool
from kynara_sdk import Kynara
kynara = Kynara.from_env()

@tool
def get_contact(contact_id: str) -> str:
    """Retrieve a CRM contact record by their ID."""
    d = kynara.check(
        subject=("agent", kynara.agent_id),
        action="crm:contacts.read",
        resource={"type": "crm.contact", "id": contact_id, "attrs": {}},
        context={"time": "10:30", "ip_country": "US"},
    )
    if d.effect == "deny":
        return f"BLOCKED by Kynara: {d.reason}"
    if d.effect == "require_approval":
        return f"PENDING approval (id={d.approval_id}) -- human must review"
    return f"Contact {contact_id}: Alice Smith | alice@acme.com | VP Eng"

@tool
def send_email(recipient: str, subject: str, body: str) -> str:
    """Send an email to a recipient."""
    d = kynara.check(
        subject=("agent", kynara.agent_id),
        action="email.send",
        resource={"type": "email", "id": recipient, "attrs": {}},
        context={"time": "10:30", "ip_country": "US"},
    )
    if d.effect != "allow":
        return f"BLOCKED by Kynara: {d.reason}"
    return f"Email sent to {recipient}: subject='{subject}'"

tools = [get_contact, send_email]
print(f"Registered {len(tools)} tools with Kynara enforcement")
```

Step 4 — create the LangChain agent and run it

```
from langchain.agents import AgentExecutor, create_react_agent
from langchain import hub
```

```

from langchain_community.llms import HuggingFaceHub

os.environ["HUGGINGFACEHUB_API_TOKEN"] = "hf_YOUR_TOKEN_HERE"
llm = HuggingFaceHub(
    repo_id="mistralai/Mistral-7B-Instruct-v0.2",
    model_kwargs={"temperature": 0.1, "max_new_tokens": 300},
)
prompt = hub.pull("hwchase17/react")
agent = create_react_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True, max_iterations=4)

result = executor.invoke({
    "input": "Look up contact c_001. Then send them an email saying hi."
})
print("=== Final answer ===")
print(result["output"])

```

1. Run the cell. The `verbose=True` output shows the agent's reasoning trace.
2. Watch: the agent calls `get_contact` (allowed) successfully, then tries `send_email` (no allow policy for `email.send` — default deny).
3. After the run, go to Kynara > Audit log, filter by your agent ID, and verify 1 allow (`crm:contacts.read`) and 1 deny (`email.send`). The Dashboard decision-volume chart shows a spike.

✓ Verbose `get_contact` returns `send_email` returns "BLOCKED by Kynara: No policy matched — trace Alice's default deny". The agent's final answer says it could not shows details, then send the email. Audit log shows 1 allow + 1 deny.

✗ "Repository not found" / HuggingFace error: `hf_` token is correct and the model is accessible on the free verify the Inference API.

✗ Both tools return contact info: check that you `email.send` allow policy. Default deny only applies if no policy have no matches.

□ This proves Kynara's key security property: it sits outside the LLM's trust boundary. No matter what the LLM decides, Kynara evaluates the structured request independently.

🔄 **RESET** Re-run freely. Each run adds 2 audit entries. No Kynara state to clean up.

Tests 1–16: browser only, no external accounts needed. Tests 17–19: Google Colab + free Kynara account. Test 20: Google Colab + free Hugging Face account (model: Mistral-7B-Instruct-v0.2). Slack (Test 12), Teams (Test 13), and PagerDuty (Test 14) require free accounts at their respective platforms.

Kynara · kynaraai.com | docs: kynaraai.com/docs | sandbox: kynaraai.com/sandbox